Name - Soham Somnath Kulkarni
Roll No - 407I081
Class - BEIT
Title - Build the Image classification model by dividing the model into following 4 stages:
a. Loading and preprocessing the image data
b. Defining the model's architecture
c. Training the model
d. Estimating the model's performance

```
import numpy as np
import pandas as pd
import random
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import accuracy_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Conv2D,Dense,MaxPooling2D
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.datasets import mnist
```

```
(X_train, y_train),(X_test, y_test) = mnist.load_data()
```

```
print(X_train.shape)
```

```
(60000, 28, 28)
```

```
X_train[0].min(),X_train[0].max()
```

```
(np.uint8(0), np.uint8(255))
```

```
X_train = (X_train - 0.0) / (255.0 - 0.0)
X_test = (X_test - 0.0) / (255.0 - 0.0)
X_train[0].min(), X_train[0].max()
(0.0, 1.0)
```

```
(0.0, 1.0)
```

```
def plot_digit(image, digit, plt, i):
    plt.subplot(4, 5, i + 1)
    plt.imshow(image, cmap=plt.get_cmap('gray'))
    plt.title(f"Digit: {digit}")
    plt.xticks([])
    plt.yticks([])
plt.figure(figsize=(16, 10))
for i in range(20):
    plot_digit(X_train[i], y_train[i], plt, i)
plt.show()
```

```
C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_12432\3073521008.py:10: User
Warning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

```
X_train = X_train.reshape((X_train.shape+ (1,)))
X_test = X_test.reshape((X_test.shape+(1,)))
```

```
y_train[0:20]
```

```
array([5, 0, 4, 1, 9, 2, 1, 3, 1, 4, 3, 5, 3, 6, 1, 7, 2, 8, 6, 9],
      dtype=uint8)
```

```
model = Sequential([
    Conv2D(32,(3,3), activation="relu",input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(100,activation="relu"),
    Dense(10,activation="softmax")
])
```

```
C:\Users\AJINKYA\AppData\Local\Temp\claude\C--Users-AJINKYA-OneDrive-Deskt
op-SUBMISSION\71b4792e-b329-4612-8d3d-b3fb5437a28d\scratchpad\venv\Lib\sit
e-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: D
o not pass an `input_shape`/`input_dim` argument to a layer. When using Se
quential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
optimizer = SGD(learning_rate=0.01, momentum=0.9)
model.compile(
    optimizer=optimizer,
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 100)	540,900
dense_1 (Dense)	(None, 10)	1,010

Total params: 542,230 (2.07 MB)

Trainable params: 542,230 (2.07 MB)

Non-trainable params: 0 (0.00 B)

```
Model_log=model.fit(X_train, y_train, epochs=10, batch_size=15, verbose=1, validation_
data=(X_test, y_test));
```

```
Epoch 1/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 4ms/step - accuracy: 0.9475 - loss: 0.1
709 - val_accuracy: 0.9788 - val_loss: 0.0606

Epoch 2/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9840 - loss: 0.05
22

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 4ms/step - accuracy: 0.9840 - loss: 0.0
522 - val_accuracy: 0.9852 - val_loss: 0.0456

Epoch 3/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 17s 4ms/step - accuracy: 0.9894 - loss: 0.0
328 - val_accuracy: 0.9858 - val_loss: 0.0432

Epoch 4/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 4ms/step - accuracy: 0.9932 - loss: 0.0
217 - val_accuracy: 0.9860 - val_loss: 0.0437

Epoch 5/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9956 - loss: 0.01
44

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 4ms/step - accuracy: 0.9956 - loss: 0.0
144 - val_accuracy: 0.9863 - val_loss: 0.0427

Epoch 6/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 4ms/step - accuracy: 0.9968 - loss: 0.0
104 - val_accuracy: 0.9872 - val_loss: 0.0404

Epoch 7/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 5ms/step - accuracy: 0.9981 - loss: 0.0
068 - val_accuracy: 0.9886 - val_loss: 0.0377

Epoch 8/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 16s 4ms/step - accuracy: 0.9988 - loss: 0.0
045 - val_accuracy: 0.9882 - val_loss: 0.0409

Epoch 9/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 18s 4ms/step - accuracy: 0.9995 - loss: 0.0
020 - val_accuracy: 0.9882 - val_loss: 0.0433

Epoch 10/10

4000/4000 ━━━━━━━━━━━━━━━━━━━━ 16s 4ms/step - accuracy: 0.9998 - loss: 0.0
011 - val_accuracy: 0.9897 - val_loss: 0.0377
```

```
plt.figure(figsize=(16, 10))
for i in range(20):
    image = random.choice(X_test).squeeze()
    digit = np.argmax(model.predict(image.reshape((1, 28, 28, 1)))[0], axis=-1)
    plot_digit(image, digit, plt, i)
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step

C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_12432\4026498666.py:6: UserW
arning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

```
predictions = np.argmax(model.predict(X_test),axis=-1)
accuracy_score(y_test,predictions)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

0.9897
```

```
n = random.randint(0,9999)
plt.imshow(X_test[n])
plt.show()
```

```
C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_12432\735217378.py:3: UserWa
rning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

```
predicted_value = model.predict(X_test)
print("Handwritten number in the image is = %d" %np.argmax(predicted_value[n]))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

Handwritten number in the image is = 6
```

```
score = model.evaluate(X_test,y_test,verbose=0)
print('Test loss:' , score[0])
print('Testaccuracy:',score[1])
```

```
Test loss: 0.03768604248762131
Testaccuracy: 0.9897000193595886
```